IoT Devices Security Improvement Based on Collaborative Intrusion Detection System and Blockchain Technology

1st Yempabou Yves Stéphane LOARI
Laboratoire de Mathématiques et d'Informatique
Université Joseph Ki-Zerbo
Ouagadougou, Burkina Faso
yveslouari@gmail.com

2nd Didier BASSOLE
Laboratoire de Mathématiques et d'Informatique
Université Joseph Ki-Zerbo
Ouagadougou, Burkina Faso
didier.bassole@ujkz.bf

3rd Leonard M. SAWADOGO
Laboratoire de Mathématiques et d'Informatique
Université Joseph Ki-Zerbo
Ouagadougou, Burkina Faso
lsawadogom@gmail.com

4th Gouayon KOALA
Laboratoire de Mathématiques et d'Informatique
Université Joseph Ki-Zerbo
Ouagadougou, Burkina Faso
gouayonkoala1@gmail.com

5th Oumarou SIÉ
Laboratoire de Mathématiques et d'Informatique
Université Joseph Ki-Zerbo
Ouagadougou, Burkina Faso
oumarou.sie@gmail.com

2024 International Conference on Computer and Applications (ICCA) | 979-8-3503-6756-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCA62237.2024.10927992

***Abstract*—IoT is one of today's major growth areas. With its low computing power and massively exchanged heterogeneous data, it is vulnerable to numerous attacks. In this paper, we proposed a collaborative system based on hybrid intrusion detection and blockchain technology to improve IoT devices security. Thus, we simulated a federated learning using Deep Neural Network, Flower library and CICIoT2023 dataset to detect and classify IoT network traffic as malicious or benign for zero-day attack whose signatures are not yet known and for those whose signatures are already known, we use signature-based IDS. The CICIoT2023 dataset used was divided into ten (10) samples using StratifiedKFold methods and each sample was assigned to a client. The results showed the effectiveness of our approach, with an accuracy around 98.8%. At last, we developed and deployed a smart contract on Solidity to simulate the dissemination of alerts between blockchain nodes.**

***Index Terms*—IoT security, federated learning, CIDS, Blockchain**

I. Introduction

The Internet of Things (IoT) is a network of interconnected objects that interact and communicate with each other in real time. These objects primarily consist of monitoring and sensory devices deployed in target physical environments, designed to seamlessly bridge the physical and digital worlds by generating continuous data streams [1]. The IoT context is characterized by low computing power, generally unmonitored networks, and the massive exchange of heterogeneous data, which introduces new security vulnerabilities [2], [3]. An Intrusion Detection System (IDS) plays a crucial role in monitoring network packets, analyzing them to detect attacks from the Internet attempting to compromise system security, as well as internal attacks seeking to misuse system resources [4]. However, in the IoT environment, IDS systems face several limitations. A few authors came up with the idea of a collaborative approach to distribute the workload among multiple IDS. While this reduces the strain on individual IDS, these solutions struggle to effectively handle zero-day attacks.

The main objective of this work is to propose an approach that draws on collaboration between anomaly-based IDS through a federated learning model, signature-based IDS and blockchain technology to enhance IoT devices security notably zero-day attack detection and classification. Then, we try to improve the performance of collaborative approache against zero-day attacks by reducing the false positive rate thanks to the federated learning model in this heterogeneous environment. The blockchain technology will at the end enhance the reliability and integrity of the system.

The remainder of this paper is organized as follows : in section II, we discuss related works. In section III, we present our methodological approach and the architecture of the proposed system. Sections IV, present the implementation as well as the results and discussions and section V concludes this work.

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

II. Related works

Many authors have proposed collaborative approaches that integrate blockchain technology to enhance the security of IoT environments:

- Erukala Suresh et al. [5] have proposed a collaborative, hybrid Blockchain-based intrusion detection system that includes a signature-based detection system (for early detection of DDoS attacks) and a collaborative behaviour-based detection system (for detecting new DDoS attacks using learning algorithms: SVM, DT, NB). The hybrid system is installed on the blockchain network and monitors all traffic. Its main limitation is that it can only detect DDoS attacks, whereas there are other attacks to which IoT devices are vulnerable.
- As for Wenjuan et al.[6], they have proposed CBSigIDS , a generic signature-based IDS framework using Blockchain to secure signature sharing against malicious nodes in an IoT environment. Unlike the previous solution, CBSigIDS is based on a signature-based detection system, limiting itself to known attacks only. The results of the experiments carried out show that a malicious person must succeed in infecting the majority of the nodes forming the blockchain in order to defeat the system. However, this secure signature exchange framework has a major limitation in that it is unable to detect new attacks due to the limited number of known signatures.
- In contrast to the previously cited work, Tomer Golomb et al.[7], have proposed a behaviour-based system. They identified (two) problems encountered by this system: the inability to detect attacks during the learning phase and the high production of false positives. This is due to the normal and rare behaviours that were not captured during the model's learning phase. As a solution, they propose that IoT devices of the same type collaborate together and simultaneously around the Blockchain to produce a more reliable global behaviour, based on the behaviours captured by each device locally. The main drawback of this approach is that for each type of IoT device, new separate chains will have to be published to store the global behaviour.
- In their work, Sudarshan and al.[8] have proposed a blockchain-based monitoring framework for the detection and isolation of malicious devices in Internet of Things (IoT) systems. Unlike previous authors, who focus on protecting the IoT environment by detecting attacks as quickly as possible, Sudarshan and his collaborators started from the premise that IoT devices are predisposed to being compromised. Their solution uses Hyperledger Blockchain technology in a monitoring framework to detect compromised devices and isolate them from the network based on a security policy. This prevents any malicious device from sending messages to all the devices on the network thanks to a digital signature system that will be used to validate the authenticity of the messages. The limitation of the system is its vulnerability to identity theft, because if a malicious device managed to steal the identity of another device by pretending to be it, it could carry out unauthorised actions. The proposed solution also faces a vulnerability issue when a large number of IoT devices are of the same type or are interconnected on the same type of add-on hardware module.
- Hafsa Benaddi et al.[9] have proposed a review of collaborative intrusion detection for IoT integrating blockchain technology. They identified that the exchange of sensitive data between IoT devices represents a major challenge, requiring several levels of security. The main cause is the exponential number of bytes of data circulating on the network every 24 hours. Faced with such a multitude of data, the question here is to think about a technology that will process this data while guaranteeing security and confidentiality in an IoT network, based on CIDS.
- Subi et al.[10] for their part identified that IoT devices have multiple use cases with varying latency requirements. In most of these use cases, IoT devices communicate their data on a centralized server (cloud), which constitutes a vulnerability in terms of data security and confidentiality. So they propose an approach based on decentralized technology such as the Ethereum blockchain. This technology will in fact serve as a secure data storage ledger, using the PoA consensus algorithm. A smart contract is developed and deployed on each node of the blockchain to manage consensus. Results have shown that as the block period (time to set up a block) of a node increases, so does its latency and computing power. This makes the implementation of blockchain in the IoT even more difficult, as these devices suffer from computational constraints. As for limitations, the proposed solution does not protect IoT devices against attacks such as DDoS, Mirai, Spoofing, etc. What's more, the creation of blocks by blockchain nodes could take a long time due to the massive volume of data that is increasingly exchanged in the IoT world.
- Nahom et al.[11] have also identified that the Internet of Things has security gaps. IDSs were introduced as a solution, but their effectiveness diminished when used in decentralised systems. This led to the development of a CIDS so that different IDS nodes could collect and share information with each other to improve performance. However, this system raised other problems such as confidentiality, trust and secure information sharing. To address these issues, Subi et al. presented a study of CIDS using blockchain technology.
- Sawadogo Leonard and al.[12] have proposed an incremental learning, characterised by multiple learning

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

cycles that can automate the adaptation of CIDS to potential web evolutions.

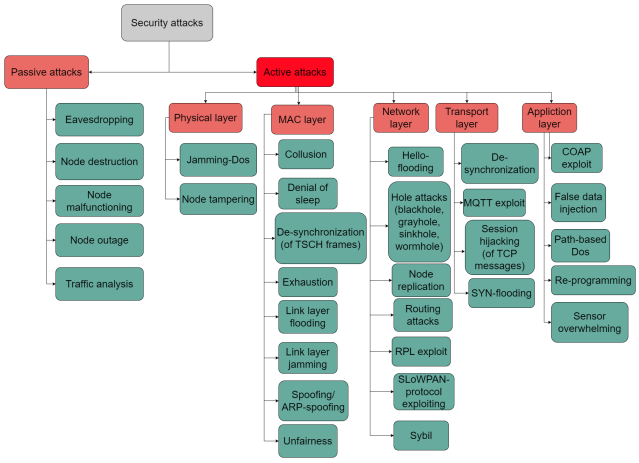


Fig. 1: Attacks on the IoT environment[13]

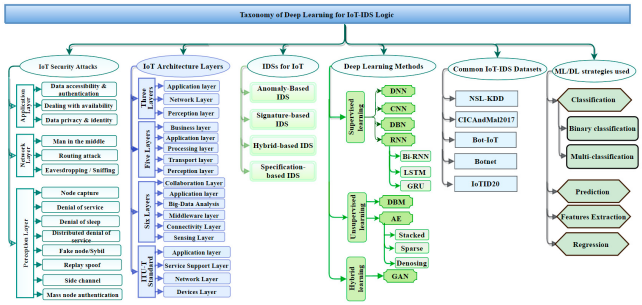


Fig. 2: Deep learning taxonomy for IoT-IDS logic[14]

A. Description of CIDS

In a decentralized collaborative intrusion detection system architecture (CIDS), each network node operates its own Intrusion Detection System (IDS) module. This design allows each node to locally analyze data and make autonomous intrusion detection decisions. As Babu et al. point out in [5], the role of an IDS is to monitor packets circulating in the network or the activities of a host, and to analyse them in order to detect attacks that attempt to compromise its security. Given the unfavourable context of the IoT environment marked by low computing power and heterogeneous data exchanged on a massive scale, authors [9],[5],[6],[7],[8] have proposed collaborative intrusion detection system. The principle is that each IDS node included in the CIDS will collaborate with all the other nodes on the basis of a certain level of trust by exchanging information about the environment. This will ultimately enable attacks to be detected more effectively, as illustrated in Fig. 3. However, the CIDS system also encounters a number of difficulties.

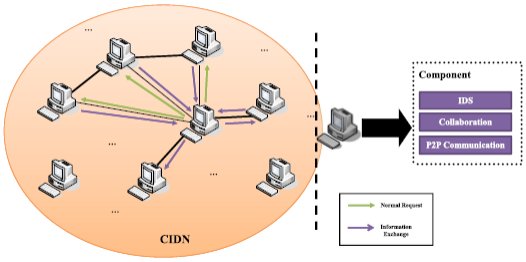


Fig. 3: CIDS architecture[5]

B. CIDS security challenges

- Data sharing: the exchange of information between CIDS nodes is a crucial issue in the context of community identification. Allowing all participants to share information can be tricky, as some entities are reluctant due to security concerns. However, without a sufficient amount of information, it becomes impossible to make progress in detection calculations and to develop a robust model for identifying malicious behaviour [5].
- Trust management: the collaborative intrusion detection system has shortcomings when faced with attacks close to the organisation. A central agent is usually responsible for accumulating information on the behaviour of nodes and traffic in order to assess trust, but this trust diminishes as the association develops, requiring a more in-depth analysis to determine the reliability of the nodes [5],[15].

C. The Blockchain as a solution

A Blockchain is a distributed transactional database, comparable to a decentralized, shared accounting ledger, which stores and transfers value or data via the Internet, transparently, securely and autonomously, as it has no central control body. This ledger is active, chronological, distributed, verifiable and protected against falsification by a system of distributed trust (consensus) between members or participants (nodes). Each member of the network has an up-to-date copy of the ledger (in near-real time), and its contents are always in phase with those of all participants [16], [17].

III. Methodological approach

A. Approach

Our approach is similar to the federated learning approach as illustrated in Fig. 4. It involves a Blockchain network made up of a collaborative intrusion detection system (CIDS), sharing attack alerts, and based on a distributed and secure architecture. To function, a blockchain needs a register, cryptography with keys to secure exchanges, an algorithm (consensus) to validate transactions, a peer-to-peer network to make it all work and a number of participants [18],[19]. The principle of our approach is as follows [16]:

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

- Initialisation of the blockchain network: it's initialised with participating nodes (IDS). A blockchain is also created to store attack alerts securely.
- Intrusion detection system (IDS): it analyse network traffic and the behaviour of IoT devices. Machine learning models are used to identify potential anomalies and attacks.
- Alert generation: when an IDS detects suspicious activity or an attack, it generates an alert. The alert can includes details such as the type of attack, the source, the nature of the malicious activity and the time of detection.
- Validation of transactions: network nodes participate in the consensus process to validate the new block. Consensus algorithms, such as Proof of Work, Proof of Stake,Byzantine Generals algorithm, guarantee agreement on the addition of the block.
- The Byzantine generals algorithm: it's a mathematical metaphor that deals with the reliability of transmissions and the integrity of interlocutors. The question is how, and to what extent, it is possible to accept information whose source or transmission channel is suspect. The solution, the algorithm of the Byzantine generals or fault tolerance, is thus the ability to tolerate machines in an arbitrary way, to allow a system to continue to function, without completely breaking down, when one of its components fails. The Byzantine generals algorithm offer advantages such as low energy consumption, resource optimization, which are favorable to the IoT environment compared to other mechanisms.
- Sharing of alerts: all the nodes in the network have access to the blockchain and can consult recorded alerts. Alerts are distributed across the entire network, ensuring common visibility of threats. Nodes can verify the authenticity of alerts by consulting the Blockchain, reinforcing confidence in the information shared.

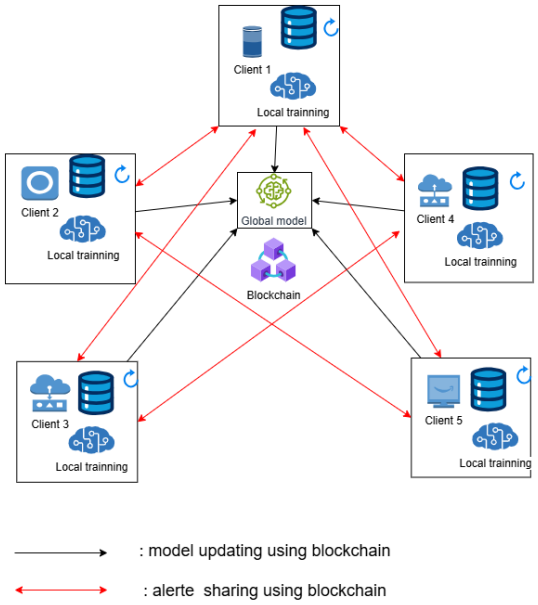


Fig. 4: Federated learning approach[19]

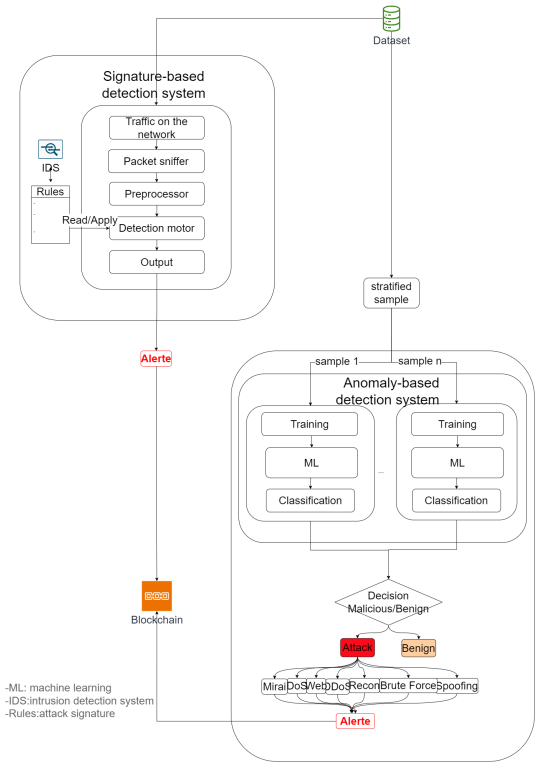


Fig. 5: Architecture of our approach

B. Architecture

The architecture, as illustrated in Fig. 5, is made up of a set of anomaly-based intrusion detection and signature-based intrusion detection. This architecture offers the advantage of combining the strengths of the two (2) types of IDS to enable the establishment of an effective collaborative system for the rapid detection of known attacks

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

and also zero-day attacks. In addition, blockchain is used to secure the exchange of alerts within this collaborative system. Signature-based IDS uses signatures describing the characteristics of different attacks. anomaly-based detection, on the other hand, relies on a machine learning algorithm to create attack profiles and benign profiles. Using these profiles, it can detect new attacks. This involves a training (or learning) phase during which it uses traffic data generally contained in a dataset [5].

IV. Implementation and tests

A. Tools

1) Google Colab: [1] Our preferred working environment was *Google Colab* a *cloud* platform that provided us with shared resources. For data processing and model implementation, we used a set of libraries and tools such as *Tensorflow*, *Pandas*, *Keras*, *Scikit-learn*, and others, contributing to the robustness and efficiency of our approach.

2) TensorFlow: [2] It is a platform dedicated to machine learning. We used it because it offers a complete and flexible ecosystem of tools, libraries and community resources for advancing machine learning. Its source code was made openly available on 9 November 2015, with version 1.0.0 officially launched on 11 February 2017. Since then, *TensorFlow* has continued to evolve, making constant improvements to its infrastructure to meet the growing needs of the *machine learning* field.

3) Pandas: [3] We have integrated the *Pandas* into our approach to data manipulation and processing. *Pandas* stands out as a tool for analysing and manipulating data. It is fast, powerful, flexible and remarkably user-friendly. It is developed using the Python programming language and offers specialised data structures as well as operations dedicated to the manipulation of numerical tables and time series. *Pandas* is open source, encouraging community collaboration and contribution.

4) Keras: [4] It is an API (Application Programming Interface) that stands out for its consistency and simplicity. It aims to minimise the number of actions required by users for common use cases. The API provides clear, actionable error messages, facilitating the development process.

5) Scikit-learn: [5] It is a free Python library dedicated to machine learning. It offers simple and effective tools for predictive data analysis.

6) NumPy: [6] It is a library written in the Python programming language, specially designed for manipulating matrices or multidimensional arrays, with mathematical functions dedicated to these data structures.

7) Flower: [7] It is an open source library that facilitates the development and deployment of federated learning mechanisms.

8) Remix: [8] It is an online integrated development environment (IDE) for creating, deploying and testing smart contracts *Ethereum*. It provides a user-friendly interface that allows Solidity developers to code, compile and deploy smart contracts directly in the web browser.

B. Tests

1) Dataset: The dataset we have used is CICIoT2023 [20]. This dataset encompasses a total of forty-seven (47) features, listed in Fig 6, with full details available in Fig 7. This comprehensive and up-to-date dataset incorporates thirty-three (33) attack types executed within an IoT topology of one hundred and five (105) devices. These attacks are rigorously classified into seven (7) main categories, namely DDoS, DoS, Recon, Web, Brute Force, Spoofing and Mirai. The inclusion of these various categories provides a thorough representation of potential attack scenarios in the IoT context.

Feature	Mean	Std	Min	25%	50%	75%	Max
flow_duration	5.76544939	285.034171	0	0	0	0.10513809	394,357.207
Header_Length	76,705.9637	461,331.747	0	54	54	280.555	9,907,147.75
Protocol Type	9.06568989	8.94553292	0	6	6	14.33	47
Duration	66.3507169	14.0191881	0	64	64	64	255
Rate	9064.05724	99,562.4906	0	2.09185589	15.7542308	117.384754	8,388,608
Srate	9064.05724	99,562.4906	0	2.09185589	15.7542308	117.384754	8,388,608
Drate	5.46×10^{-6}	0.00725077	0	0	0	0	29.7152249
fin_flag_number	0.08657207	0.28120696	0	0	0	0	1
syn_flag_number	0.20733528	0.40539779	0	0	0	0	1
rst_flag_number	0.09050473	0.28690351	0	0	0	0	1
psh_flag_number	0.08775006	0.28293106	0	0	0	0	1
ack_flag_number	0.12343168	0.32893207	0	0	0	0	1
ece_flag_number	1.48×10^{-6}	0.00121571	0	0	0	0	1
cwr_flag_number	7.28×10^{-7}	0.00085338	0	0	0	0	1
ack_count	0.09054283	0.28643144	0	0	0	0	7.7
syn_count	0.33035785	0.6635354	0	0	0	0.06	12.87
fin_count	0.09907672	0.32711642	0	0	0	0	248.32
urg_count	6.23982356	71.8524536	0	0	0	0	4401.7
rst_count	38.4681213	325.384658	0	0	0	0.01	9613
HTTP	0.04823423	0.21426079	0	0	0	0	1
HTTPS	0.05509922	0.22817383	0	0	0	0	1
DNS	0.00013068	0.01143079	0	0	0	0	1
Telnet	2.14×10^{-8}	0.00014635	0	0	0	0	1
SMTP	6.43×10^{-8}	0.00025349	0	0	0	0	1
SSH	4.09×10^{-5}	0.00639772	0	0	0	0	1
IRC	1.50×10^{-7}	0.00038722	0	0	0	0	1
TCP	0.57383427	0.49451846	0	0	1	1	1
UDP	0.21191758	0.40866676	0	0	0	0	1
DHCP	1.71×10^{-6}	0.00130903	0	0	0	0	1
ARP	6.62×10^{-5}	0.00813521	0	0	0	0	1
ICMP	0.16372157	0.37002273	0	0	0	0	1
IPv	0.99988731	0.01061485	0	1	1	1	1
LLC	0.99988731	0.01061485	0	1	1	1	1
Tot sum	1308.32257	2613.30273	42	525	567	567.54	127,335.8
Min	91.6073456	139.695326	42	50	54	54	13,583
Max	181.963418	524.030902	42	50	54	55.26	49,014
AVG	124.668815	240.991485	42	50	54	54.0497296	13,583
Std	33.3248065	160.335722	0	0	0	0.37190955	12,385.2391
Tot size	124.691567	241.549341	42	50	54	54.06	135,83
IAT	83,182,525.9	17,047,351.7	0	83,071,566	83,124,522.4	83,343,908	167,639,436
Number	9.49848933	0.81915318	1	9.5	9.5	9.5	15
Magnitue	13.12182	8.62857895	9.16515139	10	10.3923048	10.3967148	164.821115
Radius	47.0949848	226.769647	0	0	0	0.50592128	17,551.2708
Covariance	30,724.3565	323,710.68	0	0	0	1.34421569	154,902,159
Variance	0.0964376	0.233001	0	0	0	0.08	1
Weight	141.51237	21.0683073	1	141.55	141.55	141.55	244.6

Fig. 6: Description of the dataset[20]

[1] Google colab. https://colab.research.google.com
[2] Tensorflow. https://www.tensorflow.org/
[3] Pandas. https://pandas.pydata.org/
[4] Keras. https://keras.io/
[5] scikit-learn. https://scikit-learn.org/
[6] Numpy. https://numpy.org/
[7] Flower. https://flower.ai/
[8] Remix ide. https://remix.ethereum.org/

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

#	Feature	Description
1	ts	Timestamp
2	flow duration	Duration of the packet's flow
3	Header Length	Header Length
4	Protocol Type	IP, UDP, TCP, IGMP, ICMP, Unknown (Integers)
5	Duration	Time-to-Live (ttl)
6	Rate	Rate of packet transmission in a flow
7	Srate	Rate of outbound packets transmission in a flow
8	Drate,	Rate of inbound packets transmission in a flow
9	fin flag number	Fin flag value
10	syn flag number	Syn flag value
11	rst flag number	Rst flag value
12	psh flag numbe	Psh flag value
13	ack flag number	Ack flag value
14	ece flag numbe	Ece flag value
15	cwr flag number	Cwr flag value
16	ack count	Number of packets with ack flag set in the same flow
17	syn count	Number of packets with syn flag set in the same flow
18	fin count	Number of packets with fin flag set in the same flow
19	urg coun	Number of packets with urg flag set in the same flow
20	rst count	Number of packets with rst flag set in the same flow
21	HTTP	Indicates if the application layer protocol is HTTP
22	HTTPS	Indicates if the application layer protocol is HTTPS
23	DNS	Indicates if the application layer protocol is DNS
24	Telnet	Indicates if the application layer protocol is Telnet
25	SMTP	Indicates if the application layer protocol is SMTP
26	SSH	Indicates if the application layer protocol is SSH
27	IRC	Indicates if the application layer protocol is IRC
28	TCP	Indicates if the transport layer protocol is TCP
29	UDP	Indicates if the transport layer protocol is UDP
30	DHCP	Indicates if the application layer protocol is DHCP
31	ARP	Indicates if the link layer protocol is ARP
32	ICMP	Indicates if the network layer protocol is ICMP
33	IPv	Indicates if the network layer protocol is IP
34	LLC	Indicates if the link layer protocol is LLC
35	Tot sum	Summation of packets lengths in flow
36	Min	Minimum packet length in the flow
37	Max	Maximumpacket length in the flow
38	AVG	Average packet length in the flow
39	Std	Standard deviation of packet length in the flow
40	Tot size	Packet's length
41	IAT	The time difference with the previous packet
42	Number	The number of packets in the flow
43	Magnitude	(Average of the lengths of incoming packets in the flow + average of the lengths of outgoing packets in the flow)$^{0.5}$
44	Radius	(Variance of the lengths of incoming packets in the flow + variance of the lengths of outgoing packets in the flow)$^{0.5}$
45	Covariance	Covariance of the lengths of incoming and outgoing packets
46	Variance	Variance of the lengths of incoming packets in the flow / variance of the lengths of outgoing packets in the flow
47	Weight	Number of incoming packets × Number of outgoing packets

Fig. 7: CICIoT2023 features[20]

2) Execution environnement: We coded, compiled and then deployed a smart contract using Solidity, the predominant programming language on Ethereum, encompassing a set of methods specified in our system as illustrated in algorithm 1 . This will enable us to simulate the dissemination of alerts between the various nodes on the blockchain, which in this case will constitute IDSs.

Algorithm 1 Alert Dissemination

Require: $amount \leqslant$ balances[msg.sender]
  if $amount \succ$ balances[msg.sender] **then**
    Display: Insufficient amount
    return
  end if
  balances[msg.sender] $\leftarrow$ balances[msg.sender] $-$ amou
  for $i \leftarrow 0$ **to** getNumNodes() -1 **do**
    address $\leftarrow$ getNodeAddress(i)
    if address $\neq$ msg.sender **then**
      emit AlertSent(msg.sender, address)
    end if
  end for

Fig. 8 illustrates the Remix IDE environment we us This is the most widely used development environment for smart contracts on the Ethereum platform.

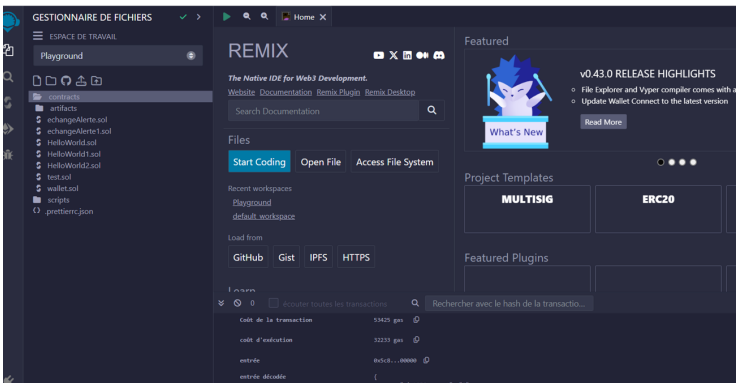


Fig. 8: Remix IDE

The training data set and the test data set were divided into 80% and 20% respectively. The training data was then divided into ten (10) samples using the StratifiedKFold method, and each sample was assigned to a client. This means that they were distributed in a stratified manner. In this way, the distribution of labels is approximately the same in each client, as shown in Fig. 9. We then used the Flower library to implement federated learning. We configured ten (10) clients to run learning on their local data. Using the Flower library, we then performed an evaluation with the test data. A DNN model architecture illustrated in the algorithm 2, for classification was used by each of the clients because of its good performance [20].

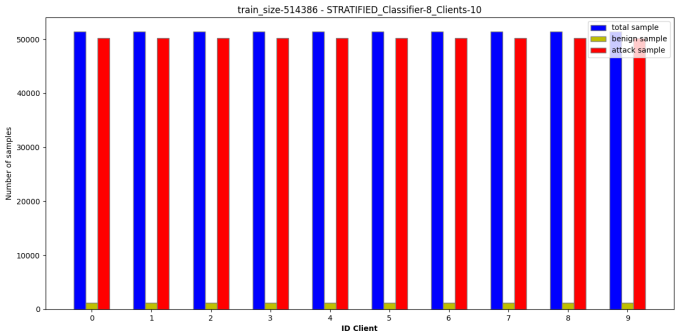


Fig. 9: Stratified data distribution

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 DNN for Attacks Classification

Input: Dataset $D = \{(x_i, y_i)\}$, where x_i are the features and y_i are the labels (0: normal, 1: DDoS, 2: DoS, 3: Mirai, 4: Recon, 5: Spoofing, 6: Web, 7: Brute Force)
Output: Classification model M capable of predicting y for a new input x
procedure MODEL CONSTRUCTION
  Step 1: Model Initialization
  $model \leftarrow \text{Sequential}()$
  Step 2: Adding the Preprocessing Layer
  $model.add(\text{Flatten}(\text{input_shape} = (\text{shape}(X_{\text{train}}[0])[1], 1)))$
  Step 3: Adding the First Hidden Layer
  $model.add(\text{Dense}(50, \text{activation} = 'relu'))$
  Step 4: Adding the Second Hidden Layer
  $model.add(\text{Dense}(25, \text{activation} = 'relu'))$
  Step 5: Adding the Output Layer
  $model.add(\text{Dense}(num_unique_classes, \text{activation} = 'softmax'))$
end procedure
procedure MODEL COMPILATION
  $model.compile(\text{optimizer} = 'adam', \text{loss} = 'categorical_crossentropy', \text{metrics} = ['accuracy'])$
end procedure
procedure MODEL TRAINING
  $model.fit(X_{\text{train}}, y_{\text{train}}, \text{epochs} = num_epochs, \text{batch_size} = 32, \text{validation_data} = (X_{\text{val}}, y_{\text{val}}))$
end procedure

3) Score and Other Measures:

- Accuracy: accuracy is defined as the ratio of correctly predicted samples to the total number of samples. The score is an appropriate measure only when the data set is balanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN} \tag{1}$$

- Precision : precision measures the proportion of instances correctly predicted as positive among all the instances predicted as positive.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

where TP (*True* Positives) is the number of instances correctly predicted as positive, and FP (*False* Positives) is the number of instances incorrectly predicted as positive.
- Recall: recall measures the proportion of instances correctly predicted as positive among all the instances that are actually positive.

$$\text{Rappel} = \frac{TP}{TP + FN} \tag{3}$$

where FN (*False Negatives*) is the number of instances that are actually positive but incorrectly predicted as negative.
- F1 score: the F1 score is the harmonic mean of precision and recall. It gives a balanced measurement between the two.

$$\text{F1} = \frac{2 * Precision * Recall}{Precision + Recall} \tag{4}$$

C. Results

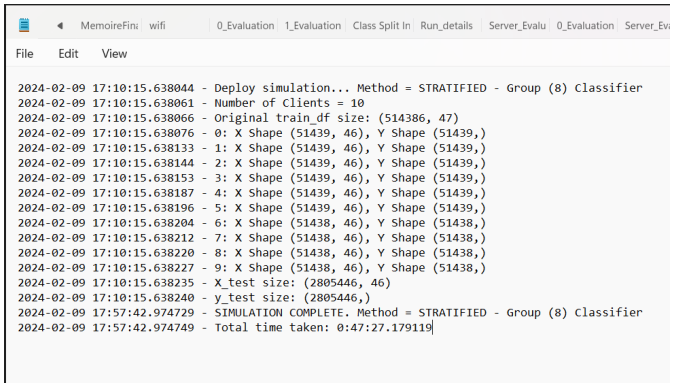
Following the previous compilation, we deployed the smart contract on one of the addresses available by default on the Remix IDE environment and simulated an alert broadcast transaction on the other addresses. This simulation was carried out by analogy with a secure exchange of alerts between participants on a Blockchain (IDS nodes). Each action on the Blockchain incurs gas charges. Gas is a measure of the amount of work done by the Ethereum network to execute a specific operation. Complex or resource-intensive transactions require more gas than simple transactions. The total cost of a transaction is calculated by multiplying gas consumption by the market-determined price of gas. According to Ethereum average gas price chart[9], the average Ethereum gas price on 16 February 2024 was thirty (30) gwei. Gas is important when deploying the smart contract and performing various functions. Users must ensure that they have sufficient gas to perform transactions on the Ethereum Blockchain. If the gas provided is not sufficient to cover the execution costs, the transaction may fail. To evaluate the execution time of transactions, in accordance with the reference Etherscan: Ethereum average gas limit chart[10] the following formula can be used:

$$\text{transaction speed} = \frac{\text{Block gas limit}}{\text{Cost of transaction gas} * \text{Block time}} \tag{5}$$

According to Etherscan: Ethereum average block time chart[11] as of February 15, 2024, the duration of a block on *Ethereum* is 12.07 seconds, while the gas limit per block is 29,999,830 units of gas.
Following the distribution of data between clients and the implementation of federated learning, we finally collected detailed information about the evaluation of each client that had trained its model locally.The results as well as the execution details were recorded in files which are presented in detail in Fig. 10 and 11.

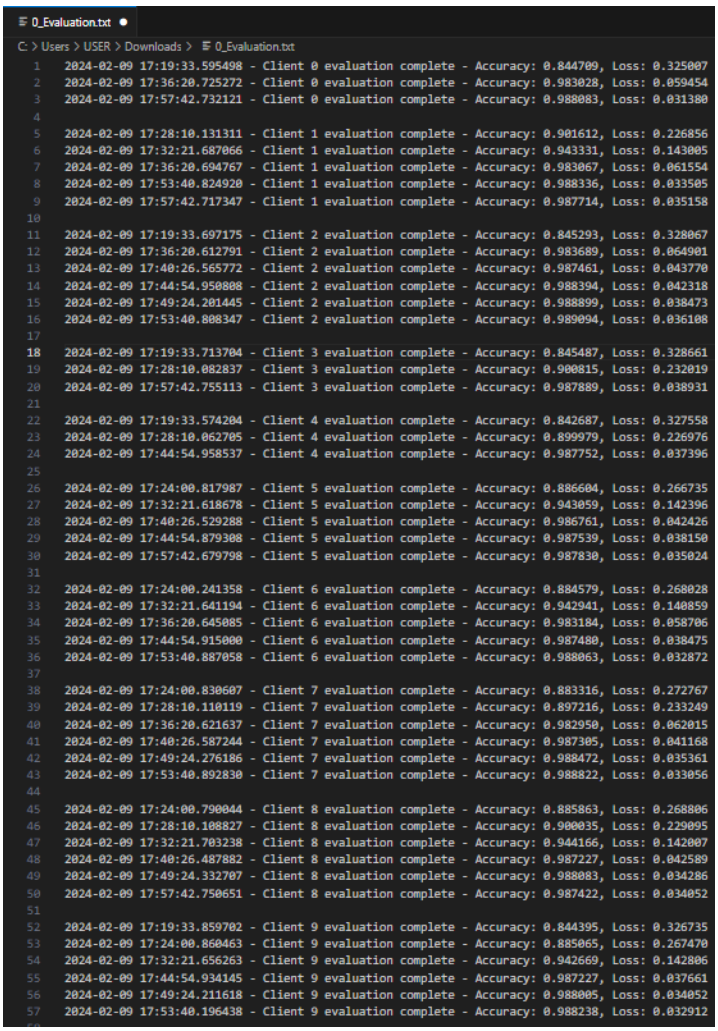
[9] https://etherscan.io/chart/gasprice
[10] https://etherscan.io/chart/gaslimit
[11] https://etherscan.io/chart/ blocktime

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.



```
2024-02-09 17:10:15.638044 - Deploy simulation... Method = STRATIFIED - Group (8) Classifier
2024-02-09 17:10:15.638061 - Number of Clients = 10
2024-02-09 17:10:15.638066 - Original train_df size: (514386, 47)
2024-02-09 17:10:15.638076 - 0: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638133 - 1: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638144 - 2: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638153 - 3: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638187 - 4: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638196 - 5: X Shape (51439, 46), Y Shape (51439,)
2024-02-09 17:10:15.638204 - 6: X Shape (51438, 46), Y Shape (51438,)
2024-02-09 17:10:15.638212 - 7: X Shape (51438, 46), Y Shape (51438,)
2024-02-09 17:10:15.638220 - 8: X Shape (51438, 46), Y Shape (51438,)
2024-02-09 17:10:15.638227 - 9: X Shape (51438, 46), Y Shape (51438,)
2024-02-09 17:10:15.638235 - X_test size: (2805446, 46)
2024-02-09 17:10:15.638240 - y_test size: (2805446,)
2024-02-09 17:57:42.974729 - SIMULATION COMPLETE. Method = STRATIFIED - Group (8) Classifier
2024-02-09 17:57:42.974749 - Total time taken: 0:47:27.179119
```

Fig. 10: Flower simulation



```
2024-02-09 17:19:33.595498 - Client 0 evaluation complete - Accuracy: 0.844709, Loss: 0.325007
2024-02-09 17:36:20.725272 - Client 0 evaluation complete - Accuracy: 0.983028, Loss: 0.059454
2024-02-09 17:57:42.732121 - Client 0 evaluation complete - Accuracy: 0.988083, Loss: 0.031380

2024-02-09 17:28:10.131311 - Client 1 evaluation complete - Accuracy: 0.901612, Loss: 0.226856
2024-02-09 17:32:21.687066 - Client 1 evaluation complete - Accuracy: 0.943331, Loss: 0.143005
2024-02-09 17:36:20.694767 - Client 1 evaluation complete - Accuracy: 0.983067, Loss: 0.061554
2024-02-09 17:53:40.824920 - Client 1 evaluation complete - Accuracy: 0.988336, Loss: 0.033505
2024-02-09 17:57:42.717347 - Client 1 evaluation complete - Accuracy: 0.987714, Loss: 0.035158

2024-02-09 17:19:33.697175 - Client 2 evaluation complete - Accuracy: 0.845293, Loss: 0.328067
2024-02-09 17:36:20.612791 - Client 2 evaluation complete - Accuracy: 0.983689, Loss: 0.064901
2024-02-09 17:40:26.565772 - Client 2 evaluation complete - Accuracy: 0.987461, Loss: 0.043770
2024-02-09 17:44:54.950808 - Client 2 evaluation complete - Accuracy: 0.988394, Loss: 0.042318
2024-02-09 17:49:24.201445 - Client 2 evaluation complete - Accuracy: 0.988899, Loss: 0.038473
2024-02-09 17:53:40.808347 - Client 2 evaluation complete - Accuracy: 0.989094, Loss: 0.036108

2024-02-09 17:19:33.713704 - Client 3 evaluation complete - Accuracy: 0.845487, Loss: 0.328661
2024-02-09 17:28:10.082837 - Client 3 evaluation complete - Accuracy: 0.900815, Loss: 0.232019
2024-02-09 17:57:42.755113 - Client 3 evaluation complete - Accuracy: 0.987889, Loss: 0.038931

2024-02-09 17:19:33.574204 - Client 4 evaluation complete - Accuracy: 0.842687, Loss: 0.327558
2024-02-09 17:28:10.062705 - Client 4 evaluation complete - Accuracy: 0.899979, Loss: 0.226976
2024-02-09 17:44:54.958537 - Client 4 evaluation complete - Accuracy: 0.987752, Loss: 0.037396

2024-02-09 17:24:00.817987 - Client 5 evaluation complete - Accuracy: 0.886604, Loss: 0.266735
2024-02-09 17:32:21.618678 - Client 5 evaluation complete - Accuracy: 0.943059, Loss: 0.142396
2024-02-09 17:40:26.529288 - Client 5 evaluation complete - Accuracy: 0.986761, Loss: 0.042426
2024-02-09 17:44:54.879308 - Client 5 evaluation complete - Accuracy: 0.987539, Loss: 0.038150
2024-02-09 17:57:42.679798 - Client 5 evaluation complete - Accuracy: 0.987830, Loss: 0.035024

2024-02-09 17:24:00.241358 - Client 6 evaluation complete - Accuracy: 0.884579, Loss: 0.268028
2024-02-09 17:32:21.641194 - Client 6 evaluation complete - Accuracy: 0.942941, Loss: 0.140859
2024-02-09 17:36:20.645085 - Client 6 evaluation complete - Accuracy: 0.983184, Loss: 0.058706
2024-02-09 17:44:54.915000 - Client 6 evaluation complete - Accuracy: 0.987480, Loss: 0.038475
2024-02-09 17:53:40.887058 - Client 6 evaluation complete - Accuracy: 0.988063, Loss: 0.032872

2024-02-09 17:24:00.830607 - Client 7 evaluation complete - Accuracy: 0.883316, Loss: 0.272767
2024-02-09 17:28:10.110119 - Client 7 evaluation complete - Accuracy: 0.897216, Loss: 0.233249
2024-02-09 17:36:20.621637 - Client 7 evaluation complete - Accuracy: 0.982950, Loss: 0.062015
2024-02-09 17:40:26.587244 - Client 7 evaluation complete - Accuracy: 0.987305, Loss: 0.041168
2024-02-09 17:49:24.276186 - Client 7 evaluation complete - Accuracy: 0.988472, Loss: 0.035361
2024-02-09 17:53:40.892830 - Client 7 evaluation complete - Accuracy: 0.988822, Loss: 0.033056

2024-02-09 17:24:00.790044 - Client 8 evaluation complete - Accuracy: 0.885863, Loss: 0.268806
2024-02-09 17:28:10.108827 - Client 8 evaluation complete - Accuracy: 0.900035, Loss: 0.229095
2024-02-09 17:32:21.703238 - Client 8 evaluation complete - Accuracy: 0.944166, Loss: 0.142007
2024-02-09 17:40:26.487882 - Client 8 evaluation complete - Accuracy: 0.987227, Loss: 0.042589
2024-02-09 17:49:24.332707 - Client 8 evaluation complete - Accuracy: 0.988083, Loss: 0.034286
2024-02-09 17:57:42.750651 - Client 8 evaluation complete - Accuracy: 0.987422, Loss: 0.034052

2024-02-09 17:19:33.859702 - Client 9 evaluation complete - Accuracy: 0.844395, Loss: 0.326735
2024-02-09 17:24:00.860463 - Client 9 evaluation complete - Accuracy: 0.885065, Loss: 0.267470
2024-02-09 17:32:21.656263 - Client 9 evaluation complete - Accuracy: 0.942669, Loss: 0.142806
2024-02-09 17:44:54.934145 - Client 9 evaluation complete - Accuracy: 0.987227, Loss: 0.037661
2024-02-09 17:49:24.211618 - Client 9 evaluation complete - Accuracy: 0.988005, Loss: 0.034052
2024-02-09 17:53:40.196438 - Client 9 evaluation complete - Accuracy: 0.988238, Loss: 0.032912
```

Fig. 11: Evaluation

Table I shows our contribution in terms of detection capacity, reduction of false positives and classification type compared to related works.

TABLE I: Comparison with related works

Ref.	Detection of multiple attacks	Reducing false positives	classification multi-classes
[5]	no	yes	no
[6]	no	yes	no
[7]	yes	no	no
Our work	**yes**	**yes**	**yes**

D. Discussion

The results obtained demonstrate the effectiveness of the proposed approach to improving the security of IoT devices through a collaborative intrusion detection system based on federated learning and Blockchain technology. The ten (10) clients evaluation shows a significant improvement in performance, with an increase in accuracy (reaching around 98.8%) and a significant reduction in losses (less than 0.04) over the iterations. These results indicate effective model convergence despite the heterogeneity of local IoT data. The homogeneity of final performance between clients assumes that the system's ability to integrate collaborative contributions to improve attack detection, particularly zero-day attacks. With regard to the Blockchain, the results also suggest the possibility of successful integration with CIDS, given that the speed of transactions was estimated at around 3 ms.

V. Conclusion

As part of our research into the security of IoT devices, we proposed improving the performance of CIDS on multiple attacks by reducing false positives, using a recent and up-to-date dataset, blockchain and a federated learning model. The results demonstrated the effectiveness of our approach, with a significant reduction in losses (less than 0.04). This offers significant benefits, such as rapid responsiveness and a reduction in false positives for the system. In terms of future prospects, it would be possible to continue the study by creating a real Blockchain network. This network would be deployed locally in a laboratory, where IoT devices would be connected, accompanied by CIDS providing security against external threats. Blockchain would play a crucial role in guaranteeing the security of the internal system through secure exchanges between CIDS.

References

[1] R Mahkeswaran, Brendan Toh, Andrew Keong Ng, and Chris Toh. Optimizing green internet of things architecture in a multi-loop aquaponics system. In *2024 IEEE 4th International Conference on Electronic Communications, Internet of Things and Big Data (ICEIB)*, pages 209–214. IEEE, 2024.

[2] Kerui Liu et al. Analysis of security vulnerabilities and threats of intelligent devices in the internet of things and countermeasures. *Journal of Electronics and Information Science*, 9(2):95–99, 2024.

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.

[3] Hitesh Mohapatra, Subhashree Rath, Subarna Panda, and Ranjan Kumar. Handling of man-in-the-middle attack in wsn through intrusion detection system. *International journal*, 8(5):1503–1510, 2020.

[4] Shakir Zaman, Haseeb Tauqeer, Wakeel Ahmad, Syed M Adnan Shah, and Muhammad Ilyas. Implementation of intrusion detection system in the internet of things: A survey. In *2020 IEEE 23rd International Multitopic Conference (INMIC)*, pages 1–6. IEEE, 2020.

[5] Erukala Suresh Babu, BKN Srinivasa Rao, M Srinivasa Rao, Ilaiah Kavati, and Ramalingaswamy Cheruku. Permissioned blockchain-based collaborative intrusion detection system to secure internet of things against ddos attacks.

[6] Wenjuan Li, Steven Tug, Weizhi Meng, and Yu Wang. Designing collaborative blockchained signature-based intrusion detection in iot environments. *Future Generation Computer Systems*, 96:481–489, 2019.

[7] Tomer Golomb, Yisroel Mirsky, and Yuval Elovici. Ciota: Collaborative iot anomaly detection via blockchain. *arXiv preprint arXiv:1803.03807*, 2018.

[8] Sreenivas Sudarshan Seshadri, David Rodriguez, Mukunda Subedi, Kim-Kwang Raymond Choo, Sara Ahmed, Qian Chen, and Junghee Lee. Iotcop: A blockchain-based monitoring framework for detection and isolation of malicious devices in internet-of-things systems. *IEEE Internet of Things Journal*, 8(5):3346–3359, 2020.

[9] Hafsa Benaddi and Khalil Ibrahimi. A review: Collaborative intrusion detection for iot integrating the blockchain technologies. In *2020 8th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, pages 1–6. IEEE, 2020.

[10] Subhi Alrubei, Jonathan Rigelsford, Callum Willis, and Edward Ball. Ethereum blockchain for securing the internet of things: practical implementation and performance evaluation. In *2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, pages 1–5. IEEE, 2019.

[11] Nahom Aron Dawit, Sujith Samuel Mathew, and Kadhim Hayawi. Suitability of blockchain for collaborative intrusion detection systems. In *2020 12th Annual Undergraduate Research Conference on Applied Computing (URC)*, pages 1–6. IEEE, 2020.

[12] Léonard M Sawadogo, Didier Bassolé, Gouayon Koala, and Oumarou Sié. Intrusions detection and classification using deep learning approach. In *Research in Computer Science and Its Applications: 11th International Conference, CNRIA 2021, Virtual Event, June 17-19, 2021, Proceedings*, pages 40–51. Springer, 2021.

[13] Ismail Butun, Patrik Österberg, and Houbing Song. Security of the internet of things: Vulnerabilities, attacks, and countermeasures. *IEEE Communications Surveys & Tutorials*, 22(1):616–644, 2019.

[14] Khalid Albulayhi, Abdallah A Smadi, Frederick T Sheldon, and Robert K Abercrombie. Iot intrusion detection taxonomy, reference architecture, and analyses. *Sensors*, 21(19):6432, 2021.

[15] Weizhi Meng, Elmar Wolfgang Tischhauser, Qingju Wang, Yu Wang, and Jinguang Han. When intrusion detection meets blockchain technology: a review. *Ieee Access*, 6:10179–10188, 2018.

[16] Vu Tuan Truong and Long Bao Le. Security for the metaverse: Blockchain and machine learning techniques for intrusion detection. *IEEE Network*, 2024.

[17] Chao Liang, Bharanidharan Shanmugam, Sami Azam, Asif Karim, Ashraful Islam, Mazdak Zamani, Sanaz Kavianpour, and Norbik Bashah Idris. Intrusion detection system for the internet of things based on blockchain and multi-agent systems. *Electronics*, 9(7):1120, 2020.

[18] Jawad Hassan, Muhammad Kamran Abid, Ali Ghulam, Muhammad Salman Fakhar, Muhammad Asif, et al. A survey on blockchain-based intrusion detection systems for iot. *VAWKUM Transactions on Computer Sciences*, 11(1):138–151, 2023.

[19] Shaashwat Agrawal, Sagnik Sarkar, Ons Aouedi, Gokul Yenduri, Kandaraj Piamrat, Mamoun Alazab, Sweta Bhattacharya, Praveen Kumar Reddy Maddikunta, and Thippa Reddy Gadekallu. Federated learning for intrusion detection system: Concepts, challenges and future directions. *Computer Communications*, 195:346–361, 2022.

[20] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A Ghorbani. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13):5941, 2023.

Authorized licensed use limited to: Nokia. Downloaded on March 28,2025 at 12:48:20 UTC from IEEE Xplore. Restrictions apply.